

8장 기본 블록과 트레이스

최광훈
전남대학교

개요

중간 표현 **Tree** 언어를 어셈블리로 번역할 때 불일치 발생

- **CJUMP** 명령어는 두 라벨 중 하나로 분기 **vs.** 어셈블리 조건 분기 명령어는 (조건이 충족되지 않으면) 다음 명령어로 분기
- 식 안에 포함된 **ESEQ** 명령어는 다루기 불편함: 식에 포함된 여러 서브 식들의 계산 순서가 다르면 최종 식의 결과가 달라지기 때문
- 식 안에 포함된 **CALL** 명령어도 동일한 문제를 일으킴
- 다른 **CALL**의 인자 식 안에 중첩된 **CALL**이 나타나면, 함수 인자를 정해진 인자 레지스터에 넣는 과정에서 내부 호출과 외부 호출이 같은 레지스터들을 겹쳐서 사용

개요

Tree 언어에서 **ESEQ**와 **2-way CJUMP**를 도입한 이유는 **AST**에서 **Tree**로 번역할 때 편리하기 때문이었음

=> **Tree**를 입력 받아 위의 문제가 없는 (의미가 동일한) **Tree**로 변환할 수 있음

=> 추가로 **SEQ**로 연결시킨 문장들을 문장들의 리스트로 변환할 수 있음

표준형 트리

- **ESEQ**와 **SEQ**를 더이상 사용하지 않음
- **CALL**은 **EXP**의 서브 트리 또는 **MOVE(TEMP t, ...)**의 서브트리로 나타남

개요

1단계: 표준형 트리(Canonical Trees)

- AST로 변환된 트리를 표준형 트리로 변환

2단계: 기본 블록 집합(A set of basic blocks)

- 기본 블록: 문장들, 중간 문장에서 분기문이나 라벨이 없음

3단계: 트레이스 집합(A set of traces)

- 모든 **CJUMP** 분기의 기본 블록 다음에 **false** 라벨로 시작하는 기본 블록으로 배치

개요

1단계: 표준형 트리(Canonical Trees)

- AST로 변환된 트리를 표준형 트리로 변환

2단계: 기본 블록 집합(A set of basic blocks)

- 기본 블록: 문장들, 중간 문장에서 분기문이나 라벨이 없음

3단계: 트레이스 집합(A set of traces)

- 모든 **CJUMP** 분기의 기본 블록 다음에 **false** 라벨로 시작하는 기본 블록으로 배치

개요

- **Linearize** 단계에서는 표현식 안에 숨어 있는 **ESEQ**를 없애고, 함수 호출인 **CALL**이 다른 표현식 내부에 중첩되지 않도록 최상위 문장 수준으로 끌어올린다.

- 그다음 **BasicBlocks** 단계에서는 분기 없이 순차적으로 실행되는 문장들을 하나의 기본 블록으로 묶는다.

- 마지막으로 **TraceSchedule** 단계에서는 조건 분기문 **CJUMP** 다음에 **false label**이 바로 오도록 기본 블록들의 실행 순서를 조정한다.

```
package Canon;
public class Canon {
    static public Tree.StmList linearize(Tree.Stm s);
}
public class BasicBlocks {
    public StmListList blocks;
    public Temp.Label done;
    public BasicBlocks(Tree.StmList stms);
}
StmListList(Tree.StmList head, StmListList tail);
public class TraceSchedule {
    public TraceSchedule(BasicBlocks b);
    public Tree.StmList stms;
}
```

8.1 표준형 트리(Canonical Trees)

(정의) 표준형 트리

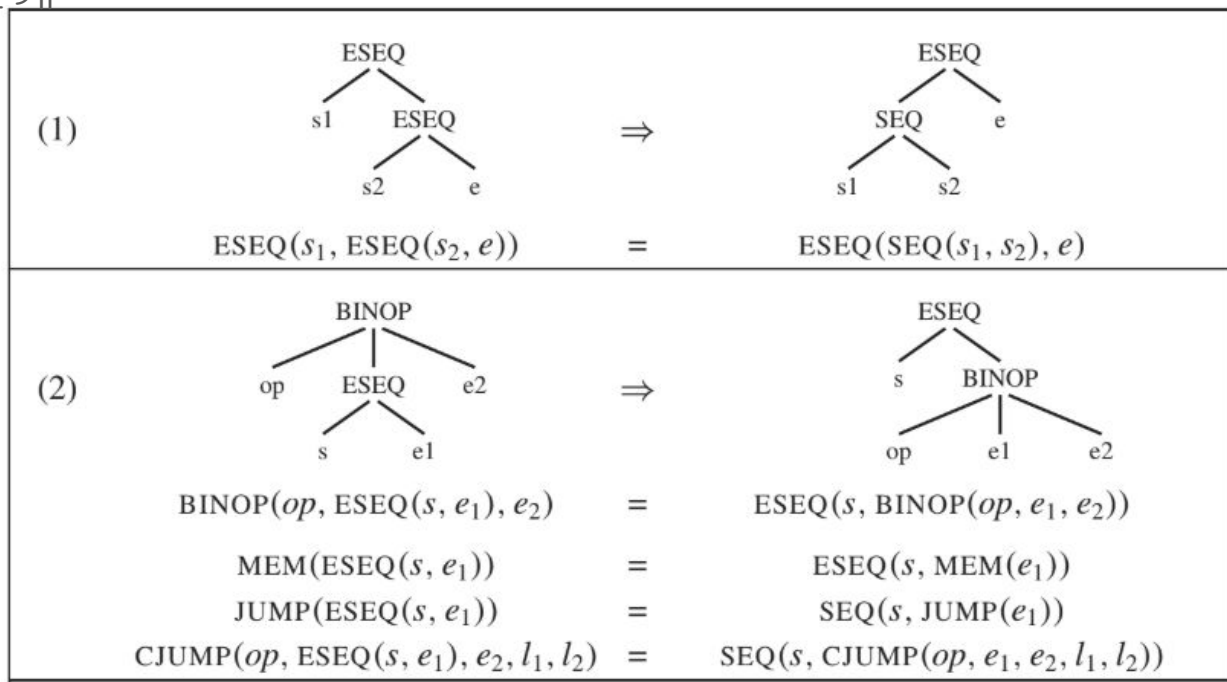
- SEQ와 ESEQ를 사용하지 않음
- CALL의 부모 노드는 EXP(..)이거나 MOVE(TEMP t, ...)

표준형 트리로 변환하는 알고리즘

- Key Idea: Figure 8.1 ESeq를 사용하는 두 트리의 동등한 관계

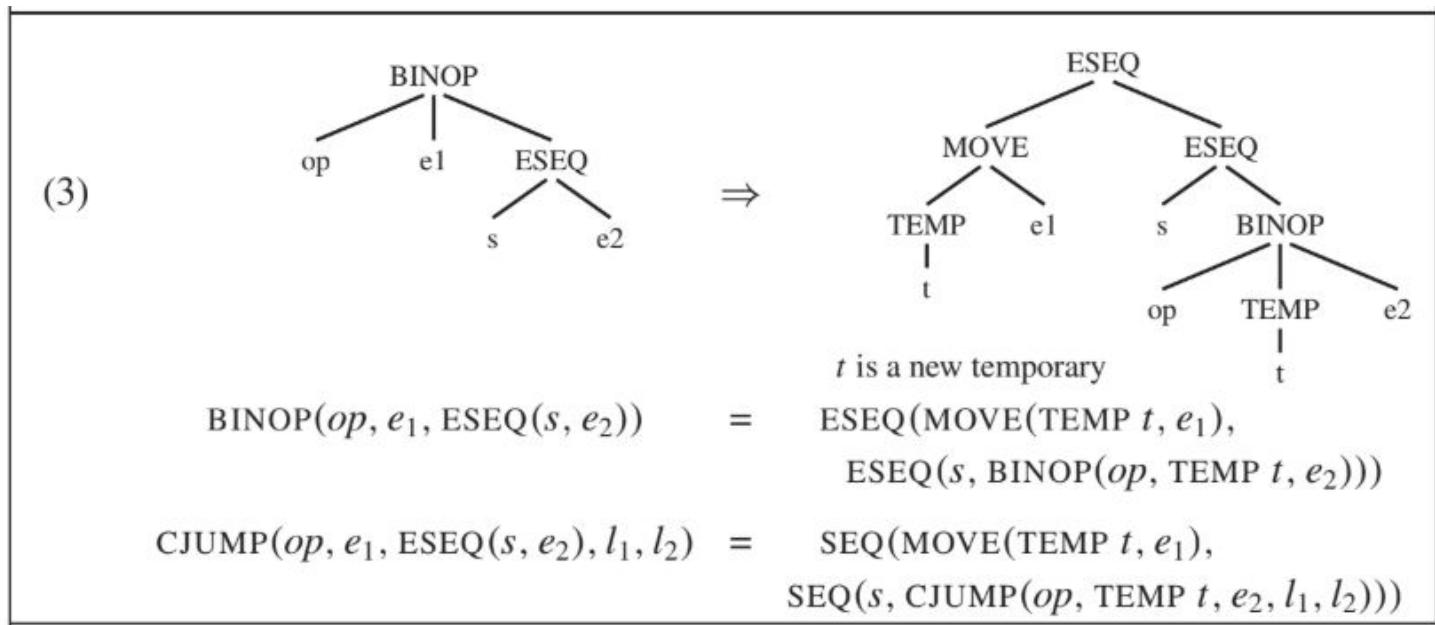
8.1 표준형 트리(Canonical Trees)

ESEQ에 관한 변환: Figure 8.1 ESeq를 사용하는 두 트리의 동등한 관계



8.1 표준형 트리(Canonical Trees)

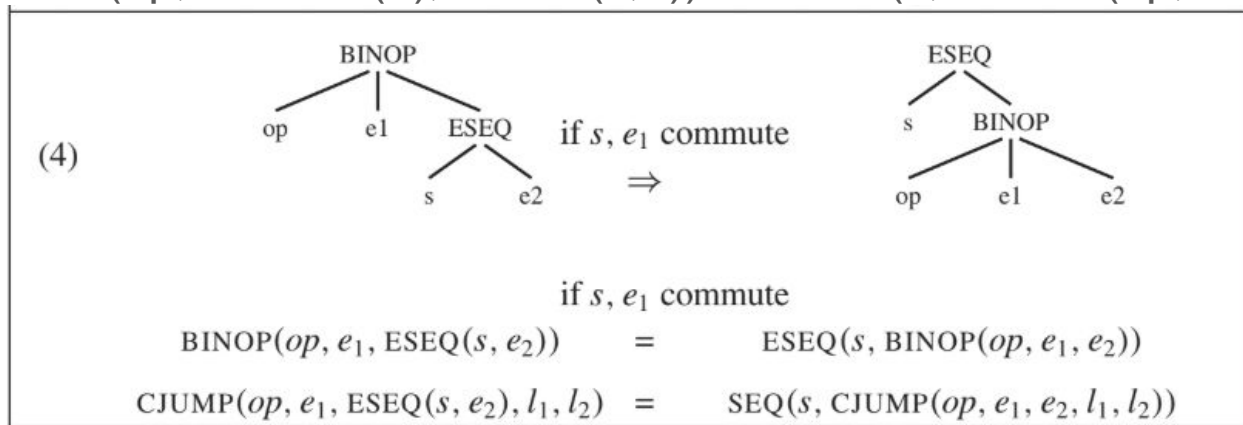
ESEQ에 관한 변환: Figure 8.1 ESeq를 사용하는 두 트리의 동등한 관계



8.1 표준형 트리(Canonical Trees)

ESEQ에 관한 변환: Figure 8.1 ESeq를 사용하는 두 트리의 동등한 관계

- $\text{BINOP}(\text{op}, \text{CONST}(n), \text{ESEQ}(s, e)) = \text{ESEQ}(s, \text{BINOP}(\text{op}, \text{CONST}(n), e))$



```
static boolean commute(Tree.Stm a, Tree.Exp b) {  
    return isNop(a)  
    || b instanceof Tree.NAME  
    || b instanceof Tree.CONST;  
}
```

```
static boolean isNop(Tree.Stm a) {  
    return a instanceof Tree.EXP  
    && ((Tree.EXP)a).exp instanceof Tree.CONST;  
}
```

8.1 표준형 트리(Canonical Trees)

CALL을 맨 상위 레벨로 올리기

- Tree 언어에서는 CALL 노드가 하위 표현식으로 사용되는 것을 허용

BINOP(PLUS, CALL(...), CALL(...))

- 실제 CALL의 구현에서는 각 함수가 자신의 결과를 동일한 전용 반환값 레지스터 TEMP(RV)에 저장하여 반환하므로, 두 번째 함수 호출이 PLUS 연산을 실행하기 전에 RV 레지스터를 덮어쓰게 됨

=> 이 문제를 해결하고자 함

8.1 표준형 트리(Canonical Trees)

CALL을 맨 상위 레벨로 올리기

- $\text{CALL}(\text{fun}, \text{args}) \Rightarrow \text{ESEQ}(\text{MOVE}(\text{TEMP } t, \text{CALL}(\text{fun}, \text{args})), \text{TEMP } t)$
- ESEQ를 제거 변환으로 $\text{MOVE}(\text{TEMP } t, \text{CALL}(\text{fun}, \text{args}))$ 를 맨 상위 레벨로 올림

8.1 표준형 트리(Canonical Trees)

문장들의 선형 리스트

- $\text{SEQ}(\text{SEQ}(a, b), c) \Leftrightarrow \text{SEQ}(a, \text{SEQ}(b, c))$
- $\text{SEQ}(s_1, \text{SEQ}(s_2, \dots, \text{SEQ}(s_{n-1}, s_n) \dots)) \Leftrightarrow s_1, s_2, \dots, s_{n-1}, s_n$

8.1 표준형 트리(Canonical Trees)

```
class Test1{  
    public static void main(String[] a){  
        // 문장 변환 예시  
        System.out.println(new Simple().add());  
    }  
}
```

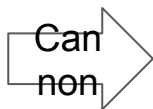
translate

1. `_heapAlloc`으로 `new Simple()` 객체 생성
2. 객체 주소를 `t0` 변수에 저장
3. `Simple_add` 함수 호출의 첫 번째 인자(`this`)로 `t0` 변수를 전달
4. 그 결과를 `_print` 함수를 호출하여 화면에 출력

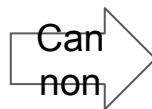
```
EXP(  
    CALL(  
        NAME _print,  
  
        CALL(  
            NAME Simple_add,  
  
            ESEQ(  
  
                MOVE(  
                    TEMP t0,  
                    CALL(  
                        NAME _heapAlloc,  
                        CONST 0)),  
  
                TEMP t0))))),
```

8.1 표준형 트리(Canonical Trees)

```
EXP(  
  CALL(  
    NAME _print,  
    CALL(  
      NAME Simple_add,  
      ESEQ(  
        MOVE(  
          TEMP t0,  
          CALL(  
            NAME _heapAlloc,  
            CONST 0)),  
          TEMP t0) ))))
```



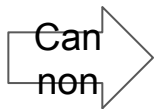
```
EXP(  
  CALL(  
    NAME _print,  
  
    ESEQ(  
      MOVE(  
        TEMP t0,  
        CALL(  
          NAME _heapAlloc,  
          CONST 0))),  
  
    CALL(  
      NAME Simple_add,  
      TEMP t0) ))
```



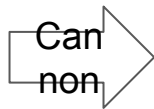
```
EXP(  
  ESEQ(  
    MOVE(  
      TEMP t0,  
      CALL(  
        NAME _heapAlloc,  
        CONST 0)),  
  
    CALL(  
      NAME _print,  
      CALL(  
        NAME Simple_add,  
        TEMP t0) )) )
```

8.1 표준형 트리(Canonical Trees)

```
EXP(  
  ESEQ(  
    MOVE(  
      TEMP t0,  
      CALL(  
        NAME _heapAlloc,  
        CONST 0)),  
  
    CALL(  
      NAME _print,  
      CALL(  
        NAME Simple_add,  
        TEMP t0)) ))
```



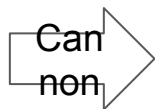
```
EXP(  
  ESEQ(  
    MOVE(  
      TEMP t0,  
      CALL(  
        NAME _heapAlloc,  
        CONST 0)),  
  
    ESEQ(  
      MOVE (  
        TEMP t1,  
        CALL(  
          NAME Simple_add,  
          TEMP t0)),  
  
      CALL(  
        NAME _print,  
        TEMP t1 )) ))
```



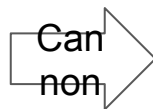
```
EXP(  
  ESEQ(  
    MOVE(  
      TEMP t0,  
      CALL(  
        NAME _heapAlloc,  
        CONST 0)),  
  
    MOVE (  
      TEMP t1,  
      CALL(  
        NAME Simple_add,  
        TEMP t0)),  
  
    CALL(  
      NAME _print,  
      TEMP t1) ))
```


8.1 표준형 트리(Canonical Trees)

```
EXP(  
  ESEQ(  
    MOVE(  
      TEMP t0,  
      CALL(  
        NAME _heapAlloc,  
        CONST 0)),  
    MOVE (  
      TEMP t1,  
      CALL(  
        NAME Simple_add,  
        TEMP t0)),  
    CALL(  
      NAME _print,  
      TEMP t1) ))
```



```
SEQ(  
  MOVE(  
    TEMP t0,  
    CALL(  
      NAME _heapAlloc,  
      CONST 0)),  
  SEQ (  
    MOVE (  
      TEMP t1,  
      CALL(  
        NAME Simple_add,  
        TEMP t0)),  
    EXP(  
      CALL(  
        NAME _print,  
        TEMP t1)) ))
```



```
[  
  MOVE(  
    TEMP t0,  
    CALL(  
      NAME _heapAlloc,  
      CONST 0)),  
  MOVE (  
    TEMP t1,  
    CALL(  
      NAME Simple_add,  
      TEMP t0)),  
  EXP(  
    CALL(  
      NAME _print,  
      TEMP t1))  
]
```

8.1 표준형 트리(Canonical Trees)

chap8/Canon/Canon.java 코드에 표준형 트리로 변환하는 알고리즘을 구현

8.2 조건 분기문 다루기

배경:

- **Tree** 언어의 **CJUMP**는 **AST**를 트리로 번역하고 분석할 때 편리하도록 두 개의 목표 레이블을 갖도록 설계됨
- 실제 기계어 조건 분기문에서는 조건이 참일 때 제어를 다른 곳으로 이동시키고 그렇지 않으면 다음 명령어로 그냥 이어서(**falls through**) 실행된다.
- 모든 **CJUMP(cond, It, If)**가 **false** 분기인 **LABEL(If)** 바로 앞에 위치하도록 트리를 재구성하여, 각 **CJUMP**를 **It**로 분기하는 기계어로 구현될 수 있도록 함

변환: 표준형 트리 문장 리스트 => 기본 블록 => 트레이스

8.2 조건 분기문 다루기: 기본 블록과 제어 흐름

제어 흐름(Control flow)이란?

- 프로그램에서 명령어들이 실행되는 순서
- 어떤 명령어 다음에 어느 명령어가 실행되는지 관심을 두고, 레지스터 값, 메모리 값, 산술 계산 결과는 무시함

조건 분기와 제어 흐름

- CJUMP(cond, true label, false label)
- 컴파일 시점에는 **cond**의 참 거짓 여부를 모르므로 둘 다 갈 수 있다고 가정

8.2 조건 분기문 다루기: 기본 블록과 제어 흐름

기본 블록이 필요한 이유?

- 분기가 아닌 명령어들은 제어 흐름 관점에서 특별한 의미가 없음
- `MOVE(...); MOVE(...); EXP(...); MOVE(...)`
- 이들을 하나의 덩어리로 묶을 수 있음 => 기본 블록(Basic Block)

기본 블록의 정의

- 첫 번째 문장은 **LABEL**, 마지막 문장은 **JUMP** 또는 **CJUMP**
- 중간에는 **LABEL**, **JUMP**, **CJUMP**가 없음

```
LABEL L1
statement
statement
statement
JUMP or CJUMP
```

8.2 조건 분기문 다루기: 기본 블록과 제어 흐름

기본 블록 생성 알고리즘

- 긴 문장 리스트를 앞에서부터 스캔
- 새 블록을 시작하는 경우: **LABEL**을 만나면 새 기본 블록 시작
- 블록을 끝내는 경우: **JUMP**나 **CJUMP**를 만나면 현재 기본 블록을 끝냄

부족한 구조 보정

- 블록이 **JUMP**나 **CJUMP**로 끝나지 않으면, 다음 블록 레이블로 분기하는 **JUMP** 추가
- 블록이 **LABEL**로 시작하지 않으면, 새 레이블을 만들어 블록 앞에 붙임

8.2 조건 분기문 다루기: 기본 블록과 제어 흐름

함수 끝과 'done' 레이블

- 함수 본문 뒤에는 보통 **epilogue**를 배치

Epilogue의 역할

- 스택 복구
- 저장된 레지스터 복구
- 호출자에게 반환

하지만 **Epilogue**는 기본 블록 목록 안에 직접 포함하지 않고, 대신 **Epilogue** 시작을 나타내는 특별한 레이블 **done**을 만들고, 마지막 블록 끝에 분기문 **JUMP(done)**을 추가

8.2 조건 분기문 다루기: 기본 블록과 제어 흐름

Canon.BasicBlocks의 기본 블록 알고리즘 구현 참고

8.2 조건 분기문 다루기: 트레이스(Traces)

기본 블록들의 순서는 어떤 순서로 배열해도 프로그램의 의미는 유지됨

- 왜냐하면 각 기본 블록의 끝에 다음 위치로 분기하는 명령어가 명시적으로 존재
- 따라서 기본 블록의 물리적 배치 순서는 바꿀 수 있음

```
LABEL b1
```

```
...
```

```
JUMP(NAME b4)
```

```
LABEL b2
```

```
...
```

```
CJUMP(cond, b3, b5)
```

8.2 조건 분기문 다루기: 트레이스(Traces)

트레이스 스케줄링의 목적

- 기본 블록의 순서를 잘 배치하면 실제 기계 코드로 번역하기 쉬움

```
...  
CJUMP(cond, true_label, false_label)  
  
LABEL false_label  
...
```

- 불필요한 JUMP 제거

```
...  
JUMP(L1)  
  
LABEL L1  
...
```

=> 결과적으로 컴파일된 코드가 더 짧고 빠르게 실행됨

8.2 조건 분기문 다루기: 트레이스(Traces)

트레이스란?

- 프로그램 실행 중에 연속해서 실행될 수 있는 기본 블록들
- 예) 제어 흐름 b1 -> b4 -> b5 트레이스 b1, b4, b6
- 트레이스 안에는 조건 분기도 포함될 수 있음

프로그램은 여러 트레이스를 가질 수 있음

- 가능한 실행 경로가 다수, 따라서 트레이스도 여러 개 존재

8.2 조건 분기문 다루기: 트레이스(Traces)

트레이스 스케줄링은 다음 조건을 만족하는 트레이스 집합을 구성함

- 모든 기본 블록이 트레이스에 포함됨
- 각 기본 블록은 정확히 하나의 트레이스에 포함됨
- 가능한 적은 수의 트레이스로 전체 프로그램을 커버함

8.2 조건 분기문 다루기: 트레이스(Traces)

조건 분기가 있는 경우: LABEL b6; ...; CJUMP(cond, b7, b3)

- 컴파일 시점에는 **cond**의 참 거짓 여부를 알 수 없어 어디로 분기할지 예측할 수 없음
- 따라서 한쪽 경로를 선택해서 트레이스를 계속 이어감
- 예) **false** 분기를 선택: ..., b6, b3, ...

8.2 조건 분기문 다루기: 트레이스(Traces)

트레이스 구성 알고리즘

- 아직 방문하지 않은 기본 블록 하나를 선택
- 이 기본 블록을 현재 트레이스의 시작점으로 정함
- 기본 블록의 분기문을 따라가며 다음 기본 블록을 트레이스에 추가
- 추가한 기본 블록은 방문했음을 표시
- 더 이상 방문하지 않은 다음 기본 블록(successor)이 없으면 트레이스를 종료
- 아직 방문하지 않은 기본 블록이 있으면 새로운 트레이스를 시작

8.2 조건 분기문 다루기: 트레이스(Traces)

트레이스 구성 예시

- b1 -> b4 -> b6
- Label b6; ...; CJUMP(cond, b7, b3)
- 트레이스1: b1, b4, b6, b3 (b6의 false 분기 b3를 트레이스에 추가)
- 트레이스2: b7

8.2 조건 분기문 다루기: 트레이스(Traces)

트레이스 구성 예시 (최적 트레이스)

<i>prologue statements</i>
JUMP(NAME test)
LABEL(<i>test</i>)
CJUMP(>, <i>i</i> , <i>N</i> , <i>done</i> , <i>body</i>)
LABEL(<i>body</i>)
<i>loop body statements</i>
JUMP(NAME test)
LABEL(<i>done</i>)
<i>epilogue statements</i>

(a)

<i>prologue statements</i>
JUMP(NAME test)
LABEL(<i>test</i>)
CJUMP($\leq$, <i>i</i> , <i>N</i> , <i>body</i> , <i>done</i>)
LABEL(<i>done</i>)
<i>epilogue statements</i>
LABEL(<i>body</i>)
<i>loop body statements</i>
JUMP(NAME test)

(b)

<i>prologue statements</i>
JUMP(NAME test)
LABEL(<i>body</i>)
<i>loop body statements</i>
JUMP(NAME test)
LABEL(<i>test</i>)
CJUMP($\leq$, <i>i</i> , <i>N</i> , <i>body</i> , <i>done</i>)
LABEL(<i>done</i>)
<i>epilogue statements</i>

(c)

FIGURE 8.3. Different trace coverings for the same program.

8.2 조건 분기문 다루기: 기본 블록과 제어 흐름

Canon.TraceSchedule의 트레이스 스케줄링 구현 참고

[실습] 연습문제 8.3

chap8/Canon/{BasicBlocks,Canon,StmListList,TraceSchedule}.java

- 제공된 Canon, BasicBlocks, TraceSchedule에 구현된 알고리즘을 설명하시오.